

0.93. Un jeu de bombes — le terrain

Un nouveau jeu, façon Bomberman. D'abord le terrain : un cadre de X, et des piliers X sur une case sur deux (colonne ET ligne paires). Le joueur, le O, en (1, 1).

1. Ce qu'il faut comprendre

Un nouveau jeu : des bombes, comme dans Bomberman. Le joueur est un **O**, les ennemis seront des **W**, les murs sont des **X**. On le construit leçon après leçon ; ici, **seulement le terrain**, et le O posé à sa place de départ.

Le terrain : un **cadre** de X tout autour (comme au 0.91.3), et à l'intérieur, des **piliers** X, un sur deux, en quadrillage. Entre les piliers, des **couloirs** : c'est là qu'on se déplacera, et que les flammes des bombes passeront.

19 colonnes, de 0 à 18, et 17 lignes, de 0 à 16. Pourquoi 19 et pas 20 ? Pour que le cadre de droite (colonne 18) soit **pair**, comme celui de gauche (colonne 0). Ainsi, il y a un couloir des deux côtés de chaque pilier. La colonne 19 reste vide ; la ligne 17 servira au score.

Ce qui est nouveau ici : une case sur deux, avec $\% 2$. $c \% 2$ est le **reste de la division par 2** (le $\%$ du 0.9) : 0 pour un nombre **pair** (0, 2, 4...), 1 pour un nombre **impair** (1, 3, 5...). Exemples : $6 \% 2 = 0$, car $6 = 3 \times 2 + 0$; $7 \% 2 = 1$, car $7 = 3 \times 2 + 1$.

Un pilier, c'est colonne paire ET ligne paire : $c \% 2 == 0 \ \&\& \ l \% 2 == 0$ ($\&\&$, « et », le 0.75). En (2, 2) : oui, pilier. En (2, 3) : la ligne 3 est impaire, non : couloir. En (3, 2) : la colonne 3 est impaire, non : couloir. En (1, 1) : ni l'une ni l'autre, c'est la place du O.

Le cadre : $l == 0 \ || \ l == 16 \ || \ c == 0 \ || \ c == 18$ ($||$, « ou », le 0.91.3) : la première ou la dernière ligne, la première ou la dernière colonne.

Deux boucles, l'une dans l'autre (le 0.69.1) : la boucle des lignes (l , de 0 à 16) contient celle des colonnes (c , de 0 à 18). Pour **chaque** ligne, on passe sur **toutes** les colonnes : $17 \times 19 = 323$ cases, une par une. Déroulons : $l = 0$: $c = 0, 1, 2 \dots 18$, tout est cadre. $l = 1$: $c = 0$ cadre, 1 à 17 rien (ligne impaire), 18 cadre. $l = 2$: $c = 0$ cadre, 1 rien, 2 pilier, 3 rien, 4 pilier... 18 cadre.

Sur le cadre, certaines cases sont aussi des piliers (par exemple (0, 0), ou (4, 16) : colonne et ligne paires). Le X y est posé deux fois : ce n'est pas grave, c'est le même X au même endroit.

Le compte : 68 X pour le cadre (19 en haut, 19 en bas, 15 à gauche, 15 à droite) et 56 piliers à l'intérieur (8 colonnes paires, de 2 à 16, $\times$ 7 lignes paires, de 2 à 14) : **124 X**.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **314 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage	– la mise en route de la console
main()	117 o	← démarrage	– ta fonction
VBlank	8 o	← la console, à chaque image	– compte les images, 60 fois par seconde
AttendreImage	28 o	← image()	– attend l'image suivante
AttendreVBlank	16 o	← démarrage	– attend que l'écran soit libre
AdresseCase	10 o	← poser()	– trouve l'adresse d'une case de l'écran
CopierTuiles	40 o	← démarrage	– copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage	– vide les deux cartes de l'écran
Tuiles	32 o	← CopierTuiles	– 2 lettres sur 43

Lettres gravées : 0X

Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, EffacerCases, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

poser() × 3	32 o	sur place, dans main()	+ AdresseCase (10 o)
image() × 1	3 o	sur place, dans main()	+ AttendreImage (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- UN JEU DE BOMBES : le terrain ----
//
//  colonne : 0 1 2 3 4 ...          18
//  ligne 0 : X X X X X X X ... X X X X X      le cadre
//  ligne 1 : X 0                      X      une ligne impaire : un couloir
//  ligne 2 : X  X  X  X  ...  X  X      une ligne paire : des piliers
//  ligne 3 : X                      X
//  ...
//  ligne 16: X X X X X X X ... X X X X X      le cadre
//
// ALPHABET[23] : le X (la 24e lettre, on compte depuis 0).
// ALPHABET[14] : le 0 (la 15e lettre).

#include <poser>      // pose une tuile sur une case du fond
#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // Deux boucles, l'une dans l'autre : chaque LIGNE, et dans chaque ligne,
    // chaque COLONNE. 17 lignes × 19 colonnes = 323 cases.
    for (uint8_t l = 0; l < 17; l++) {
        for (uint8_t c = 0; c < 19; c++) {

            // Le cadre : la première ou la dernière ligne, OU (||) la première
            // ou la dernière colonne.
            if (l == 0 || l == 16 || c == 0 || c == 18) {
                poser(c, l, ALPHABET[23]);
            }

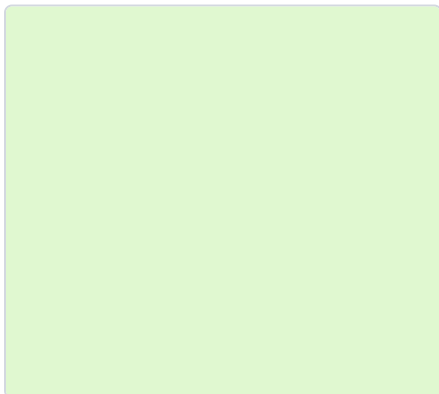
            // NOUVEAU : les piliers. c % 2 : le reste de c divisé par 2.
            // 0 si c est PAIR (0, 2, 4...), 1 s'il est IMPAIR (1, 3, 5...).
            // Colonne paire ET (&&) ligne paire : un pilier.
            // (2, 2) : pilier    (2, 3) : couloir    (3, 2) : couloir
            if (c % 2 == 0 && l % 2 == 0) {
                poser(c, l, ALPHABET[23]);
            }
        }
    }

    // Le joueur, le 0, dans le coin en haut à gauche : (1, 1).
    poser(1, 1, ALPHABET[14]);

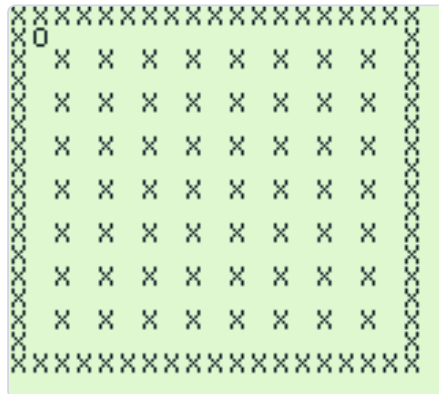
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un cadre de X, des piliers X en quadrillage à l'intérieur, et le O en haut à gauche.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le cadre : lignes 0 et 16, colonnes 0 et 18

✓ la colonne 19 reste vide

✓ la ligne 2 : un pilier sur deux

✓ les lignes impaires sont des couloirs

✓ 124 X en tout (124)

✓ le O en (1, 1)

Ensuite : **0.93.1. Un jeu de bombes — le O bouge, les X l'arrêtent** — chapitre 0 : Avant tout. · [gameboy3](#) — écrire une cartouche Game Boy en C++